Introduction

Translations

- 日本語:
  - https://t-cool.github.io/anki-addon-docs-ja/
  - http://rs.luminousspice.com/ankiaddons21/

Overview

Anki's UI is primarily written in Python/PyQt. A number of screens, such as the review screen and editor, also make use of TypeScript and Svelte. To write add-ons, you will need some basic programming experience, and some familiarity with Python. The Python tutorial is a good place to start.

Add-ons in Anki are implemented as Python modules, which Anki loads at startup. They can register themselves to be notified when certain actions take place (eg, a hook that runs when the browse screen is loaded), and can make changes to the UI (e.g.adding a new menu item) when those actions take place.

There is a brief overview of Anki's architecture available.

While it is possible to develop Anki add-ons with just a plain text editor, you can make your life much easier by using a proper code editor/IDE. Please see the Editor Setup section for more information.

Support

This document contains some hints to get you started, but it is not a comprehensive guide. To actually write an add-on, you will need to familiarize yourself with Anki's source code, and the source code of other add-ons that do similiar things to what you are trying to accomplish.

Because of our limited resources, **no official support is available for add-on writing**. If you have any questions, you will either need to find the answers yourself in the source code, or post your questions on the development forum.

You can also use the add-on forum to request someone write an add-on for you. You may need to offer some money before anyone becomes interested in helping you.

Editor Setup

While you can write an add-on with a basic text editor such as Notepad, setting up a proper Python editor/development environment (IDE) will make your life considerably easier.

PyCharm setup

The free community edition of PyCharm has good out of the box support for Python: https://www.jetbrains.com/pycharm/. You can also use other editors like Visual Studio Code, but we find PyCharm gives the best results.

Over the last year, Anki's codebase has been updated to add type hints to almost all of the code. These type hints make development easier, by providing better code completion, and by catching errors using tools like mypy. As an add-on author, you can take advantage of this type hinting as well.

To get started with your first add-on:

- Open PyCharm and create a new project.
- Right click/ctrl+click on your project on the left and create a new Python package called "myaddon"

Now you'll need to fetch Anki's bundled source code so you can get type completion. As of Anki 2.1.24, these are available on PyPI. **You will need to be using a 64 bit version of Python, and your Python version must match a version the Anki version you are fetching supports.** To install Anki via PyCharm, click on Python Console in the bottom left and type the following in:

```
import subprocess

subprocess.check_call(["pip3", "install", "--upgrade", "pip"])
subprocess.check_call(["pip3", "install", "mypy", "aqt[qt6]"])
```

Hit enter and wait. Once it completes, you should now have code completion.

If you get an error, you are probably not using a 64 bit version of Python, or your Python version is not one the latest Anki version supports. Try running the commands above with "-vvv" to get more info.

After installing, try out the code completion by double clicking on the `__init__.py` file. If you see a spinner down the bottom, wait for it to complete. Then type in:

```
from anki import hooks
hooks.
```

and you should see completions pop up.

Please note that you can not run your add-on from within PyCharm - you will get errors. Add-ons need to be run from within Anki, which is covered in the A Basic Add-on section.

MyPy

Using MyPy

The type hints you installed when setting up PyCharm can also be used to check your code is correct, using a tool called MyPy. My Py will catch some cases where you've called Anki functions incorrectly, such as when you've typed a function name in incorrectly, or passed a string when an integer was expected.

In PyCharm, click on Terminal in the bottom left, and type `mypy myaddon`. After some processing, it will show a success or tell you any mistakes you've made. For example, if you specified a hook incorrectly:

```
from aqt import gui_hooks

def myfunc() -> None:
  print("myfunc")

gui_hooks.reviewer_did_show_answer.append(myfunc)
```

Then mypy will report:

```
myaddon/__init__.py:5: error: Argument 1 to "append" of "list" has incompatible type
"Callable[[], Any]"; expected "Callable[[Card], None]"
Found 1 error in 1 file (checked 1 source file)
```

..which is telling you that the hook expects a function which takes a card as the first argument, eg

```
from anki.cards import Card

def myfunc(card: Card) -> None:
  print("myfunc")
```

Checking Existing Add-Ons

Mypy has a "check_untyped_defs" option that will give you some type checking even if your own code lacks type hints, but to get the most out of it, you will need to add type hints to your own code. This can take some initial time, but pays off in the long term, as it becomes easier to navigate your own code, and allows you to catch errors in parts of the code you might not regularly exercise yourself. It is also makes it easier to check for any problems caused by updating to a newer Anki version.

If you have a large existing add-on, you may wish to look into tools like monkeytype to automatically add types to your code.

▶ Monkeytype

Here are some example add-ons that use type hints:

https://github.com/ankitects/anki-addons/blob/master/demos/

Add-on Folders

You can access the top level add-ons folder by going to the Tools>Add-ons menu item in the main Anki window. Click on the View Files button, and a folder will pop up. If you had no add-ons installed, the top level add-ons folder will be shown. If you had an add-on selected, the add-on's module folder will be shown, and you will need to go up one level.

The add-ons folder is named "addons21", corresponding to Anki 2.1. If you have an "addons" folder, it is because you have previously used Anki 2.0.x.

Each add-on uses one folder inside the add-on folder. Anki looks for a file called `__init__.py` file inside the folder, eg:

```
addons21/myaddon/__init__.py
```

If `__init__.py` does not exist, Anki will ignore the folder.

When choosing a folder name, it is recommended to stick to a-z and 0-9 characters to avoid problems with Python's module system.

While you can use whatever folder name you wish for folders you create yourself, when you download an add-on from AnkiWeb, Anki will use the item's ID as the folder name, such as:

```
addons21/48927303923/__init__.py
```

Anki will also place a meta.json file in the folder, which keeps track of the original add-on name, when it was downloaded, and whether it's enabled or not.

You should not store user data in the add-on folder, as it's deleted when the user upgrades an add-on.

If you followed the steps in the editor setup section, you can either copy your myaddon folder into Anki's add-on folder to test it, or on Mac or Linux, create a symlink from the folder's original location into your add-ons folder.

A Basic Add-on

Add the following to `myaddon/__init__.py` in your add-ons folder:

```
# import the main window object (mw) from aqt
from aqt import mw
# import the "show info" tool from utils.py
from aqt.utils import showInfo, qconnect
# import all of the Qt GUI library
from aqt.qt import *

# We're going to add a menu item below. First we want to create a function to
# be called when the menu item is activated.

def testFunction() -> None:
    # get the number of cards in the current collection, which is stored in
    # the main window
    cardCount = mw.col.card_count()
    # show a message box
    showInfo("Card count: %d" % cardCount)

# create a new menu item, "test"
action = QAction("test", mw)
# set it to call testFunction when it's clicked
qconnect(action.triggered, testFunction)
# and add it to the tools menu
mw.form.menuTools.addAction(action)
```

Restart Anki, and you should find a 'test' item in the tools menu. Running it will display a dialog with the card count.

If you make a mistake when entering in the plugin, Anki will show an error message on startup indicating where the problem is.

The 'anki' Module

All access to your collection and associated media go through a Python package called `anki`, located in pylib/anki in Anki's source repo.

The Collection

All operations on a collection file are accessed via a `Collection` object. The currently-open Collection is accessible via a global `mw.col`, where `mw` stands for `main window`. When using the `anki` module outside of Anki, you will need to create your own Collection object.

Some basic examples of what you can do follow. Please note that you should put these in something like testFunction(). You can't run them directly in an add-on, as add-ons are initialized during Anki startup, before any collection or profile has been loaded.

Also please note that accessing the collection directly can lead to the UI temporarily freezing if the operation doesn't complete quickly - in practice you would typically run the code below in a background thread.

Get a due card:

```
card = mw.col.sched.getCard()
if not card:
    # current deck is finished
```

Answer the card:

```
mw.col.sched.answerCard(card, ease)
```

Edit a note (append " new" to the end of each field):

```
note = card.note()
for (name, value) in note.items():
    note[name] = value + " new"
mw.col.update_note(note)
```

Get card IDs for notes with tag x:

```
ids = mw.col.find_cards("tag:x")
```

Get question and answer for each of those ids:

```
for id in ids:
    card = mw.col.get_card(id)
    question = card.question()
    answer = card.answer()
```

Make reviews due tomorrow

```
ids = mw.col.find_cards("is:due")
mw.col.sched.set_due_date(ids, "1")
```

Import a text file into the collection

Requires Anki 2.1.55+.

```
from anki.collection import ImportCsvRequest
from aqt import mw
col = mw.col
path = "/home/dae/foo.csv"
metadata = col.get_csv_metadata(path=path, delimiter=None)
request = ImportCsvRequest(path=path, metadata=metadata)
response = col.import_csv(request)
print(response.log.found_notes, list(response.log.updated), list(response.log.new))
```

Almost every GUI operation has an associated function in anki, so any of the operations that Anki makes available can also be called in an add-on.

Reading/Writing Objects

Most objects in Anki can be read and written via methods in pylib.

```
card = col.get_card(card_id)
card.ivl += 1
col.update_card(card)


note = col.get_note(note_id)
note["Front"] += " hello"
col.update_note(note)


deck = col.decks.get(deck_id)
deck["name"] += " hello"
col.decks.save(deck)

deck = col.decks.by_name("Default hello")
...


config = col.decks.get_config(config_id)
config["new"]["perDay"] = 20
col.decks.save(config)


notetype = col.models.get(notetype_id)
notetype["css"] += "\nbody { background: grey; }\n"
col.models.save(note)

notetype = col.models.by_name("Basic")
...
```

You should prefer these methods over directly accessing the database, as they take care of marking items as requiring a sync, and they prevent some forms of invalid data from being written to the database.

For locating specific cards and notes, col.find_cards() and col.find_notes() are useful.

The Database

:warning: You can easily cause problems by writing directly to the database. Where possible, please use methods such as the ones mentioned above instead.

Anki's DB object supports the following functions:

scalar() returns a single item:

```
showInfo("card count: %d" % mw.col.db.scalar("select count() from cards"))
```

list() returns a list of the first column in each row, e.g.[1, 2, 3]:

```
ids = mw.col.db.list("select id from cards limit 3")
```

all() returns a list of rows, where each row is a list:

```
ids_and_ivl = mw.col.db.all("select id, ivl from cards")
```

execute() can also be used to iterate over a result set without building an intermediate list. eg:

```
for id, ivl in mw.col.db.execute("select id, ivl from cards limit 3"):
    showInfo("card id %d has ivl %d" % (id, ivl))
```

execute() allows you to perform an insert or update operation. Use named arguments with ?. eg:

```
mw.col.db.execute("update cards set ivl = ? where id = ?", newIvl, cardId)
```

Note that these changes won't sync, as they would if you used the functions mentioned in the previous section.

executemany() allows you to perform bulk update or insert operations. For large updates, this is much faster than calling execute() for each data point. eg:

```
data = [[newIvl1, cardId1], [newIvl2, cardId2]]
mw.col.db.executemany(same_sql_as_above, data)
```

As above, these changes won't sync.

Add-ons should never modify the schema of existing tables, as that may break future versions of Anki.

If you need to store addon-specific data, consider using Anki's Configuration support.

If you need the data to sync across devices, small options can be stored within mw.col.conf. Please don't store large amounts of data there, as it's currently sent on every sync.

Command-Line Use

The `anki` module can be used separately from Anki's GUI. It is strongly recommended you use it instead of attempting to read or write a .anki2 file directly.

Install it with pip:

```
$ pip install anki
```

Then you can use it in a .py file, like so:

```
from anki.collection import Collection
col = Collection("/path/to/collection.anki2")
print(col.sched.deck_due_tree())
```

See the Anki module for more.

Hooks & Filters

- New Style Hooks
- Notable Hooks
  - Webview
    - Managing External Resources in Webviews
- Legacy Hook Handling
- Adding Hooks

Hooks are the way you should connect your add-on code to Anki. If the function you want to alter doesn't already have a hook, please see the section below about adding new hooks.

There are two different kinds of "hooks":

- Regular hooks are functions that don't return anything. They are run for their side effects, and may sometimes alter the objects they have been passed, such as inserting an extra item in a list.

- "Filters" are functions that return their first argument, after maybe changing it. An example filter is one that takes the text of a field during card display, and returns an altered version.

The distinction is necessary because some data types in Python can be modified directly, and others can only be modified by creating a changed copy (such as strings).

New Style Hooks

A new style of hook was added in Anki 2.1.20.

Imagine you wish to show a message each time the front side of a card is shown in the review screen. You've looked at the source code in reviewer.py, and seen the following line in the showQuestion() function:

```
gui_hooks.reviewer_did_show_question(card)
```

To register a function to be called when this hook is run, you can do the following in your add-on:

```
from aqt import gui_hooks

def myfunc(card):
  print("question shown, card question is:", card.q())

gui_hooks.reviewer_did_show_question.append(myfunc)
```

Multiple add-ons can register for the same hook or filter - they will all be called in turn.

To remove a hook, use code like:

```
gui_hooks.reviewer_did_show_question.remove(myfunc)
```

:warning: Functions you attach to a hook should not modify the hook while they are executing, as it will break things:

```
def myfunc(card):
  # DON'T DO THIS!
  gui_hooks.reviewer_did_show_question.remove(myfunc)

gui_hooks.reviewer_did_show_question.append(myfunc)
```

An easy way to see all hooks at a glance is to look at pylib/tools/genhooks.py and qt/tools/genhooks_gui.py.

If you have set up type completion as described in an earlier section, you can also see the hooks in your IDE:

0:00 / 0:26

In the above video, holding the command/ctrl key down while hovering will show a tooltip, including arguments and documentation if it exists. The argument names and types for the callback can be seen on the bottom line.

For some examples of how the new hooks are used, please see https://github.com/ankitects/anki-addons/blob/master/demos/.

Most of the new style hooks will also call the legacy hooks (described further below), so old add-ons will continue to work for now, but add-on authors are encouraged to update to the new style as it allows for code completion, and better error checking.

Notable Hooks

For a full list of hooks, and their documentation, please see

- The GUI hooks
- The pylib hooks

Webview

Many of Anki's screens are built with one or more webviews, and there are some hooks you can use to intercept their use.

From Anki 2.1.22:

- `gui_hooks.webview_will_set_content()` allows you to modify the HTML that various screens send to the webview. You can use this for adding your own HTML/CSS/Javascript to particular screens. This will not work for external pages - see the Anki 2.1.36 section below.
- `gui_hooks.webview_did_receive_js_message()` allows you to intercept messages sent from Javascript. Anki provides a `pycmd(string)` function in Javascript which sends a message back to Python, and various screens such as reviewer.py respond to the messages. By using this hook, you can respond to your own messages as well.

From Anki 2.1.36:

- `webview_did_inject_style_into_page()` gives you an opportunity to inject styling or content into external pages like the graphs screen and congratulations page that are loaded with load_ts_page().

Managing External Resources in Webviews

Add-ons may expose their own web assets by utilizing `aqt.addons.AddonManager.setWebExports()`. Web exports registered in this manner may then be accessed under the `/_addons` subpath.

For example, to allow access to a `my-addon.js` and `my-addon.css` residing in a "web" subfolder in your add-on package, first register the corresponding web export:

```
from aqt import mw
mw.addonManager.setWebExports(__name__, r"web/.*(css|js)")
```

Then, append the subpaths to the corresponding web_content fields within a function subscribing to `gui_hooks.webview_will_set_content`:

```
def on_webview_will_set_content(web_content: WebContent, context) -> None:
    addon_package = mw.addonManager.addonFromModule(__name__)
    web_content.css.append(f"/_addons/{addon_package}/web/my-addon.css")
    web_content.js.append(f"/_addons/{addon_package}/web/my-addon.js")
```

Note that '/' will also match the os specific path separator.

Legacy Hook Handling

Older versions of Anki used a different hook system, using the functions runHook(), addHook() and runFilter().

For example, when the scheduler (anki/sched.py) discovers a leech, it calls:

```
runHook("leech", card)
```

If you wished to perform a special operation when a leech was discovered, such as moving the card to a "Difficult" deck, you could do it with the following code:

```
from anki.hooks import addHook
from aqt import mw

def onLeech(card):
    # can modify without .flush(), as scheduler will do it for us
    card.did = mw.col.decks.id("Difficult")
    # if the card was in a cram deck, we have to put back the original due
    # time and original deck
    card.odid = 0
    if card.odue:
        card.due = card.odue
        card.odue = 0

addHook("leech", onLeech)
```

An example of a filter is in aqt/editor.py. The editor calls the "editFocusLost" filter each time a field loses focus, so that add-ons can apply changes to the note:

```
if runFilter(
    "editFocusLost", False, self.note, self.currentField):
    # something updated the note; schedule reload
    def onUpdate():
        self.loadNote()
        self.checkValid()
    self.mw.progress.timer(100, onUpdate, False)
```

Each filter in this example accepts three arguments: a modified flag, the note, and the current field. If a filter makes no changes it returns the modified flag the same as it received it; if it makes a change it returns True. In this way, if any single add-on makes a change, the UI will reload the note to show updates.

The Japanese Support add-on uses this hook to automatically generate one field from another. A slightly simplified version is presented below:

```
def onFocusLost(flag, n, fidx):
    from aqt import mw
    # japanese model?
    if "japanese" not in n.model()['name'].lower():
        return flag
    # have src and dst fields?
    for c, name in enumerate(mw.col.models.fieldNames(n.model())):
        for f in srcFields:
            if name == f:
                src = f
                srcIdx = c
        for f in dstFields:
            if name == f:
                dst = f
    if not src or not dst:
        return flag
    # dst field already filled?
    if n[dst]:
        return flag
    # event coming from src field?
    if fidx != srcIdx:
        return flag
    # grab source text
    srcTxt = mw.col.media.strip(n[src])
    if not srcTxt:
        return flag
    # update field
    try:
        n[dst] = mecab.reading(srcTxt)
    except Exception, e:
        mecab = None
        raise
    return True

addHook('editFocusLost', onFocusLost)
```

The first argument of a filter is the argument that should be returned. In the focus lost filter this is a flag, but in other cases it may be some other object. For example, in anki/collection.py, _renderQA() calls the "mungeQA" filter which contains the generated HTML for the front and back of cards. latex.py uses this filter to convert text in LaTeX tags into images.

In Anki 2.1, a hook was added for adding buttons to the editor. It can be used like so:

```
from aqt.utils import showInfo
from anki.hooks import addHook

# cross out the currently selected text
def onStrike(editor):
    editor.web.eval("wrap('<del>', '</del>');")

def addMyButton(buttons, editor):
    editor._links['strike'] = onStrike
    return buttons + [editor._addButton(
        "iconname", # "/full/path/to/icon.png",
        "strike", # link name
        "tooltip")]

addHook("setupEditorButtons", addMyButton)
```

Adding Hooks

If you want to modify a function that doesn't already have a hook, please submit a pull request that adds the hooks you need.

In your PR, please describe the use-case you're trying to solve. Hooks that are general in nature will typically be approved; hooks that target a very specific use case may need to be refactored to be more general first. For an example of what this might look like, please see this PR.

The hook definitions are located in pylib/tools/genhooks.py and qt/tools/genhooks_gui.py. When building Anki, the build scripts will automatically update the hook files with the definitions listed there.

Please see the docs/ folder in the source tree for more information.

Console Output

Because Anki is a GUI app, text output to stdout (e.g. `print("foo")`) is not usually visible to the user. You can optionally reveal text printed to stdout, and it is recommended that you do so while developing your add-on.

Warnings

Anki uses stdout to print warnings about API deprecations, eg:

```
addons21/mytest/__init__.py:10:getNote is deprecated: please use 'get_note'
```

If these warnings are occurring in a loop, please address them promptly, as they can slow Anki down even if the console is not shown.

Printing text

You may find it useful to print text to stdout to aid in debugging your add-on. Please avoid printing large amounts of text (e.g.in a loop that deals with hundreds or thousands of items), as that may slow Anki down, even if the console is not shown.

Showing the Console

Windows

If you start Anki via the `anki-console.bat` file in `C:\Users\user\AppData\Local\Programs\Anki` (or `C:\Program Files\Anki`), a separate console window will appear.

macOS

Open Terminal.app, then enter the following text and hit enter:

```
/Applications/Anki.app/Contents/MacOS/anki
```

Linux

Open a terminal/xterm, then run Anki with `anki`

Background Operations

If your add-on performs a long-running operation directly, the user interface will freeze until the operation completes - no progress window will be shown, and the app will look as if it's stuck. This is annoying for users, so care should be taken to avoid it happening.

The reason it happens is because the user interface runs on the "main thread". When your add-on performs a long-running operation directly, it also runs on the main thread, and it prevents the UI code from running again until your operation completes. The solution is to run your add-on code in a background thread, so that the UI can continue to function.

A complicating factor is that any code you write that interacts with the UI also needs to be run on the main thread. If your add-on only ran in the background, and it attempted to access the UI, it would cause Anki to crash. So selectivity is required - UI operations should be run on the main thread, and long-running operations like collection and network access should be run in the background. Anki provides some tools to make this easier.

Read-Only/Non-Undoable Operations

For long-running operations like gathering a group of notes, or things like network access, `QueryOp` is recommended. For the latter, make sure to read about serialization further below.

In the following example, my_ui_action() will return quickly, and the operation will continue to run in the background until it completes. If it finishes successfully, on_success will be called.

```
from anki.collection import Collection
from aqt.operations import QueryOp
from aqt.utils import showInfo
from aqt import mw

def my_background_op(col: Collection, note_ids: list[int]) -> int:
    # some long-running op, eg
    for id in note_ids:
        note = col.get_note(note_id)
        # ...

    return 123

def on_success(count: int) -> None:
    showInfo(f"my_background_op() returned {count}")

def my_ui_action(note_ids: list[int]):
    op = QueryOp(
        # the active window (main window in this case)
        parent=mw,
        # the operation is passed the collection for convenience; you can
        # ignore it if you wish
        op=lambda col: my_background_op(col, note_ids),
        # this function will be called if op completes successfully,
        # and it is given the return value of the op
        success=on_success,
    )

    # if with_progress() is not called, no progress window will be shown.
    # note: QueryOp.with_progress() was broken until Anki 2.1.50
    op.with_progress().run_in_background()
```

Be careful not to directly call any Qt/UI routines inside the background operation!

- If you need to modify the UI after an operation completes (e.g.show a tooltip), you should do it from the success function.
- If the operation needs data from the UI (e.g.a combo box value), that data should be gathered prior to executing the operation.
- If you need to update the UI during the background operation (e.g.to update the text of the progress window), your operation needs to perform that update on the main thread. For example, in a loop:

```
if time.time() - last_progress >= 0.1:
    aqt.mw.taskman.run_on_main(
        lambda: aqt.mw.progress.update(
            label=f"Remaining: {remaining}",
            value=total - remaining,
            max=total,
        )
    )
    last_progress = time.time()
```

Operations are serialized by default

By default, only a single operation can run at once, to ensure multiple read operations on the collection don't interleave with another write operation.

If your operation does not touch the collection (e.g., it is a network request), then you can opt out of this serialization so that the operation runs concurrently to other ops:

```
op.without_collection().run_in_background()
```

Collection Operations

A separate `CollectionOp` is provided for undoable operations that modify the collection. It functions similarly to QueryOp, but will also update the UI as changes are made (e.g.refresh the Browse screen if any notes are changed).

Many undoable ops already have a `CollectionOp` defined in aqt/operations/*.py. You can often use one of them directly rather than having to create your own. For example:

```
from aqt.operations.note import remove_notes

def my_ui_action(note_ids: list[int]) -> None:
    remove_notes(parent=mw, note_ids=note_ids).run_in_background()
```

By default that routine will show a tooltip on success. You can call .success() or .failure() on it to provide an alternative routine.

For more information on undo handling, including combining multiple operations into a single undo step, please see this forum page.

Qt and PyQt

As mentioned in the overview, Anki uses PyQt for a lot of its UI, and the Qt documentation and PyQt documentation are invaluable for learning how to display different GUI widgets.

Qt Versions

From Anki 2.1.50, separate builds are provided for PyQt5 and PyQt6. Generally speaking, if you write code that works in Qt6, and make sure to import any Qt classes from aqt.qt instead of directly from PyQt6, your code should also work in Qt5.

Designer Files

Parts of Anki's UI are defined in .ui files, located in `qt/aqt/forms`. Anki's build process converts them into .py files. If you wish to build your add-on's UI in a similar way, you will need to install Python, and install a program called Qt Designer (Designer.app on macOS). On Linux, it may be available in your distro's packages; on Windows and Mac, you'll need to install it as part of a Qt install. Once installed, you will need to use a program provided in the pyqt6 pip package to compile the .ui files.

Generated Python files for PyQt6 won't work with PyQt5 and vice versa, so if you wish to support both versions, you will need to build the .ui files twice, once with pyuic5, and once with pyuic6.

Garbage Collection

One particular thing to bear in mind is that objects are garbage collected in Python, so if you do something like:

```
def myfunc():
    widget = QWidget()
    widget.show()
```

...then the widget will disappear as soon as the function exits. To prevent this, assign top level widgets to an existing object, like:

```
def myfunc():
    mw.myWidget = widget = QWidget()
    widget.show()
```

This is often not required when you create a Qt object and give it an existing object as the parent, as the parent will keep a reference to the object.

Python Modules

From Anki 2.1.50, the packaged builds include most built-in Python modules. Earlier versions ship with only the standard modules necessary to run Anki.

If your add-on uses a standard Python module that has not been included, or a package from PyPI, then your add-on will need to bundle the module.

For pure Python modules, this is usually as simple as putting them in a subfolder, and adjusting sys.path. For modules that require C extensions such as numpy, things get a fair bit more complicated, as you'll need to bundle the different module versions for each platform, and ensure you're bundling a version that is compatible with the version of Python Anki is packaged with.

Add-on Config

Config JSON

Add-ons can store config data in a JSON dictionary. You provide the default values by shipping a file called `config.json`. A simple example:

```
{"myvar": 5}
```

In config.md:

```
This is documentation for this add-on's configuration, in *markdown* format.
```

In your add-on's code:

```
from aqt import mw
config = mw.addonManager.getConfig(__name__)
print("var is", config['myvar'])
```

If the config hasn't been customized, the default values from that file will be used.

If you need to programmatically modify the config, you can save your changes with:

```
mw.addonManager.writeConfig(__name__, config)
```

Users are also able to edit the config inside the GUI.

The edited config is stored in `meta.json`.

When `getConfig()` is used after edits, meta.json is used preferentially. If a key is missing from meta.json's config, Anki will fall back on the default config.

If you change the value of existing keys in config.json, users who have customized their configuration will continue to see the old values unless they use the "restore defaults" button.

If no config.json file exists, getConfig() will return None - even if you have called writeConfig().

Add-ons that manage options in their own GUI can have that GUI displayed when the config button is clicked:

```
mw.addonManager.setConfigAction(__name__, myOptionsFunc)
```

Avoid key names starting with an underscore - they are reserved for future use by Anki.

User Files

When your add-on needs configuration data other than simple keys and values, it can use a special folder called user_files in the root of your add-on's folder. Any files placed in this folder will be preserved when the add-on is upgraded. All other files in the add-on folder are removed on upgrade.

To ensure the user_files folder is created for the user, you can put a README.txt or similar file inside it before zipping up your add-on.

When Anki upgrades an add-on, it will ignore any files in the .zip that already exist in the user_files folder.

Reviewer Javascript

For a general solution not specific to card review, see the webview section.

Anki provides a hook to modify the question and answer HTML before it is displayed in the review screen, preview dialog, and card layout screen. This can be useful for adding Javascript to the card. If you wish to load external resources in your card, please see managing external resources in webviews.

An example:

```
from aqt import gui_hooks
def prepare(html, card, context):
    return html + """
<script>
document.body.style.background = "blue";
</script>"""
gui_hooks.card_will_show.append(prepare)
```

The hook takes three arguments: the HTML of the question or answer, the current card object (so you can limit your add-on to specific note types for example), and a string representing the context the hook is running in.

Make sure you return the modified HTML.

Context is one of: "reviewQuestion", "reviewAnswer", "clayoutQuestion", "clayoutAnswer", "previewQuestion" or "previewAnswer".

The answer preview in the card layout screen, and the previewer set to "show both sides" will only use the "Answer" context. This means Javascript you append on the back side of the card should not depend on Javascript that is only added on the front.

Because Anki fades the previous text out before revealing the new text, Javascript hooks are required to perform actions like scrolling at the correct time. You can use them like so:

```
from aqt import gui_hooks
def prepare(html, card, context):
    return html + """
<script>
onUpdateHook.push(function () {
    window.scrollTo(0, 2000);
})
</script>"""
gui_hooks.card_will_show.append(prepare)
```

- onUpdateHook fires after the new card has been placed in the DOM, but before it is shown.
- onShownHook fires after the card has faded in.

The hooks are reset each time the question or answer is shown.

Debugging

Exceptions and Stdout/Stderr

If your code throws an uncaught exception, it will be caught by Anki's standard exception handler, and an error will be presented to the user.

The handler catches anything that is printed to stderr, so you should avoid logging text to stderr unless you want the user to see it in a popup.

Text printed to standard output is covered in this section.

Webviews

If you set the env var QTWEBENGINE_REMOTE_DEBUGGING to 8080 prior to starting Anki, you can surf to http://localhost:8080 in Chrome to debug the visible webpages.

Alternatively, you can use this add-on to open the inspector inside Anki.

Debug Console

Anki also includes a REPL. From within the program, press the shortcut key and a window will open up. You can enter expressions or statements into the top area, and then press ctrl+return/command+return to evaluate them. An example session follows:

```
>>> mw
<no output>

>>> print(mw)
<aqt.main.AnkiQt object at 0x10c0ddc20>

>>> invalidName
Traceback (most recent call last):
  File "/Users/dae/Lib/anki/qt/aqt/main.py", line 933, in onDebugRet
    exec text
  File "<string>", line 1, in <module>
NameError: name 'invalidName' is not defined

>>> a = [a for a in dir(mw.form) if a.startswith("action")]
... print(a)
... print()
... pp(a)
['actionAbout', 'actionCheckMediaDatabase', ...]

['actionAbout',
 'actionCheckMediaDatabase',
 'actionDocumentation',
 'actionDonate',
 ...]

>>> pp(mw.reviewer.card)
<anki.cards.Card object at 0x112181150>

>>> pp(card()) # shortcut for mw.reviewer.card.__dict__
{'_note': <anki.notes.Note object at 0x11221da90>,
 '_qa': [...]
 'col': <anki.collection._Collection object at 0x1122415d0>,
 'data': u'',
 'did': 1,
 'due': -1,
 'factor': 2350,
 'flags': 0,
 'id': 1307820012852L,
 [...]
}

>>> pp(bcard()) # shortcut for selected card in browser
<as above>
```

Note that you need to explicitly print an expression in order to see what it evaluates to. Anki exports pp() (pretty print) in the scope to make it easier to quickly dump the details of objects, and the shortcut ctrl+shift+return will wrap the current text in the upper area with pp() and execute the result.

PDB

If you're on Linux or are running Anki from source, it's also possible to debug your script with pdb. Place the following line somewhere in your code, and when Anki reaches that point it will kick into the debugger in the terminal:

```
from aqt.qt import debug; debug()
```

Alternatively you can export DEBUG=1 in your shell and it will kick into the debugger on an uncaught exception.

Python Assertions

Runtime checks using Python's `assert` statement are not evaluated in Anki's release builds, even when running in debug mode. If you want to use `assert` for testing you can use the packaged versions from PyPI or run Anki from source.

Monkey Patching and Method Wrapping

If you want to modify a function that doesn't already have a hook, it's possible to overwrite that function with a custom version instead. This is sometimes referred to as 'monkey patching'.

Monkey patching is useful in the testing stage, and while waiting for new hooks to be integrated into Anki. But please don't rely on it long term, as monkey patching is very fragile, and will tend to break as Anki is updated in the future.

The only exception to the above is if you're making extensive changes to Anki where adding new hooks would be impractical. In that case, you may unfortunately need to modify your add-on periodically as Anki is updated.

In aqt/editor.py there is a function setupButtons() which creates the buttons like bold, italics and so on that you see in the editor. Let's imagine you want to add another button in your add-on.

Anki 2.1 no longer uses setupButtons(). The code below is still useful to understand how monkey patching works, but for adding buttons to the editor please see the setupEditorButtons hook described in the previous section.

The simplest way is to copy and paste the function from the Anki source code, add your text to the bottom, and then overwrite the original, like so:

```
from aqt.editor import Editor

def mySetupButtons(self):
    <copy & pasted code from original>
    <custom add-on code>

Editor.setupButtons = mySetupButtons
```

This approach is fragile however, as if the original code is updated in a future version of Anki, you would also have to update your add-on. A better approach would be to save the original, and call it in our custom version:

```
from aqt.editor import Editor

def mySetupButtons(self):
    origSetupButtons(self)
    <custom add-on code>

origSetupButtons = Editor.setupButtons
Editor.setupButtons = mySetupButtons
```

Because this is a common operation, Anki provides a function called wrap() which makes this a little more convenient. A real example:

```
from anki.hooks import wrap
from aqt.editor import Editor
from aqt.utils import showInfo

def buttonPressed(self):
    showInfo("pressed " + `self`)

def mySetupButtons(self):
    # - size=False tells Anki not to use a small button
    # - the lambda is necessary to pass the editor instance to the
    #   callback, as we're passing in a function rather than a bound
    #   method
    self._addButton("mybutton", lambda s=self: buttonPressed(self),
                    text="PressMe", size=False)

Editor.setupButtons = wrap(Editor.setupButtons, mySetupButtons)
```

By default, wrap() runs your custom code after the original code. You can pass a third argument, "before", to reverse this. If you need to run code both before and after the original version, you can do so like so:

```
from anki.hooks import wrap
from aqt.editor import Editor

def mySetupButtons(self, _old):
    <before code>
    ret = _old(self)
    <after code>
    return ret

Editor.setupButtons = wrap(Editor.setupButtons, mySetupButtons, "around")
```

Sharing Add-ons

- Sharing via AnkiWeb
- Sharing outside AnkiWeb

Sharing via AnkiWeb

You can package up an add-on for distribution by zipping it up, and giving it a name ending in .ankiaddon.

The top level folder should not be included in the zip file. For example, if you have a module like the following:

```
addons21/myaddon/__init__.py
addons21/myaddon/my.data
```

Then the zip file contents should be:

```
__init__.py
my.data
```

If you include the folder name in the zip like the following, AnkiWeb will not accept the zip file:

```
myaddon/__init__.py
myaddon/my.data
```

On Unix-based machines, you can create a properly-formed file with the following command:

```
$ cd myaddon && zip -r ../myaddon.ankiaddon *
```

Python automatically creates `pycache` folders when your add-on is run. Please make sure you delete these prior to creating the zip file, as AnkiWeb can not accept zip files that contain `pycache` folders.

Once you've created a .ankiaddon file, you can use the Upload button on https://ankiweb.net/shared/addons/ to share the add-on with others.

Sharing outside AnkiWeb

If you wish to distribute .ankiaddon files outside of AnkiWeb, your add-on folder needs to contain a 'manifest.json' file. The file should contain at least two keys: 'package' specifies the folder name the add-on will be stored in, and 'name' specifies the name that will be shown to the user. You can optionally include a 'conflicts' key which is a list of other packages that conflict with the add-on, and a 'mod' key which specifies when the add-on was updated.

When Anki downloads add-ons from AnkiWeb, only the conflicts key is used from the manifest.

Porting 2.1.x Add-ons

Please see https://forums.ankiweb.net/t/porting-tips-for-anki-23-10/35916

Porting Anki 2.0 add-ons

- Python 3
- Qt5 / PyQt5
- Single .py add-ons need their own folder
- Folders are deleted when upgrading
- Supporting both 2.0 and 2.1 in one codebase
- Webview Changes
- Reviewer Changes
- Add-on Configuration

Python 3

Anki 2.1 requires Python 3 or later. After installing Python 3 on your machine, you can use the 2to3 tool to automatically convert your existing scripts to Python 3 code on a folder by folder basis, like:

```
2to3-3.8 --output-dir=aqt3 -W -n aqt
mv aqt aqt-old
mv aqt3 aqt
```

Most simple code can be converted automatically, but there may be parts of the code that you need to manually modify.

Qt5 / PyQt5

The syntax for connecting signals and slots has changed in PyQt5. Recent PyQt4 versions support the new syntax as well, so the same syntax can be used for both Anki 2.0 and 2.1 add-ons.

More info is available at http://pyqt.sourceforge.net/Docs/PyQt4/new_style_signals_slots.html

One add-on author reported that the following tool was useful to automatically convert the code: https://github.com/rferrazz/pyqt4topyqt5

The Qt modules are in 'PyQt5' instead of 'PyQt4'. You can do a conditional import, but an easier way is to import from aqt.qt - eg

```
from aqt.qt import *
```

That will import all the Qt objects like QDialog without having to specify the Qt version.

Single .py add-ons need their own folder

Each add-on is now stored in its own folder. If your add-on was previously called `demo.py`, you'll need to create a `demo` folder with an `__init__.py` file.

If you don't care about 2.0 compatibility, you can just rename `demo.py` to `demo/__init__.py`.

If you plan to support 2.0 with the same file, you can copy your original file into the folder (`demo.py` → `demo/demo.py`), and then import it relatively by adding the following to `demo/__init__.py`:

```
from . import demo
```

The folder needs to be zipped up when uploading to AnkiWeb. For more info, please see sharing add-ons.

Folders are deleted when upgrading

When an add-on is upgraded, all files in the add-on folder are deleted. The only exception is the special user_files folder. If your add-on requires more than simple key/value configuration, make sure you store the associated files in the user_files folder, or they will be lost on upgrade.

Supporting both 2.0 and 2.1 in one codebase

Most Python 3 code will run on Python 2 as well, so it is possible to update your add-ons in such a way that they run on both Anki 2.0 and 2.1. Whether this is worth it depends on the changes you need to make.

Most add-ons that affect the scheduler should require only minor changes to work on 2.1. Add-ons that alter the behaviour of the reviewer, browser or editor may require more work.

The most difficult part is the change from the unsupported QtWebKit to QtWebEngine. If you do any non-trivial work with webviews, some work will be required to port your code to Anki 2.1, and you may find it difficult to support both Anki versions in the one codebase.

If you find your add-on runs without modification, or requires only minor changes, you may find it easiest to add some if statements to your code and upload the same file for both 2.0.x and 2.1.x.

If your add-on requires more significant changes, you may find it easier to stop providing updates for 2.0.x, or to maintain separate files for the two Anki versions.

Webview Changes

Qt 5 has dropped WebKit in favour of the Chromium-based WebEngine, so Anki's webviews are now using WebEngine. Of note:

- You can now debug the webviews using an external Chrome instance, by setting the env var QTWEBENGINE_REMOTE_DEBUGGING to 8080 prior to starting Anki, then surfing to localhost:8080 in Chrome.
- WebEngine uses a different method of communicating back to Python. AnkiWebView() is a wrapper for webviews which provides a pycmd(str) function in Javascript which will call the ankiwebview's onBridgeCmd(str) method. Various parts of Anki's UI like reviewer.py and deckbrowser.py have had to be modified to use this.
- Javascript is evaluated asynchronously, so if you need the result of a JS expression you can use ankiwebview's evalWithCallback().
- As a result of this asynchronous behaviour, editor.saveNow() now requires a callback. If your add-on performs actions in the browser, you likely need to call editor.saveNow() first and then run the rest of your code in the callback. Calls to .onSearch() will need to be changed to .search()/.onSearchActivated() as well. See the browser's .deleteNotes() for an example.
- Various operations that were supported by WebKit like setScrollPosition() now need to be implemented in javascript.
- Page actions like mw.web.triggerPageAction(QWebEnginePage.Copy) are also asynchronous, and need to be rewritten to use javascript or a delay.
- WebEngine doesn't provide a keyPressEvent() like WebKit did, so the code that catches shortcuts not attached to a menu or button has had to be changed. setStateShortcuts() fires a hook that can be used to adjust the shortcuts for a given state.

Reviewer Changes

Anki now fades the previous card out before fading the next card in, so the next card won't be available in the DOM when the showQuestion hook fires. There are some new hooks you can use to run Javascript at the appropriate time - see here for more.

Add-on Configuration

Many small 2.0 add-ons relied on users editing the sourcecode to customize them. This is no longer a good idea in 2.1, because changes made by the user will be overwritten when they check for and download updates. 2.1 provides a Configuration system to work around this. If you need to continue supporting 2.0 as well, you could use code like the following:

```
if getattr(getattr(mw, "addonManager", None), "getConfig", None):
    config = mw.addonManager.getConfig(__name__)
else:
    config = dict(optionA=123, optionB=456)
```